

3559. Number of Ways to Assign Edge Weights II

Solved

Hard

Topics

Companies

Hint

There is an undirected tree with n nodes labeled from 1 to n , rooted at node 1. The tree is represented by a 2D integer array `edges` of length $n - 1$, where `edges[i] = [ui, vi]` indicates that there is an edge between nodes `ui` and `vi`.

Initially, all edges have a weight of 0. You must assign each edge a weight of either 1 or 2.

The **cost** of a path between any two nodes `u` and `v` is the total weight of all edges in the path connecting them.

You are given a 2D integer array `queries`. For each `queries[i] = [ui, vi]`, determine the number of ways to assign weights to edges **in the path** such that the cost of the path between `ui` and `vi` is **odd**.

Return an array `answer`, where `answer[i]` is the number of valid assignments for `queries[i]`.

Since the answer may be large, apply **modulo** $10^9 + 7$ to each `answer[i]`.

Note: For each query, disregard all edges **not** in the path between node `ui` and `vi`.

Example 1:



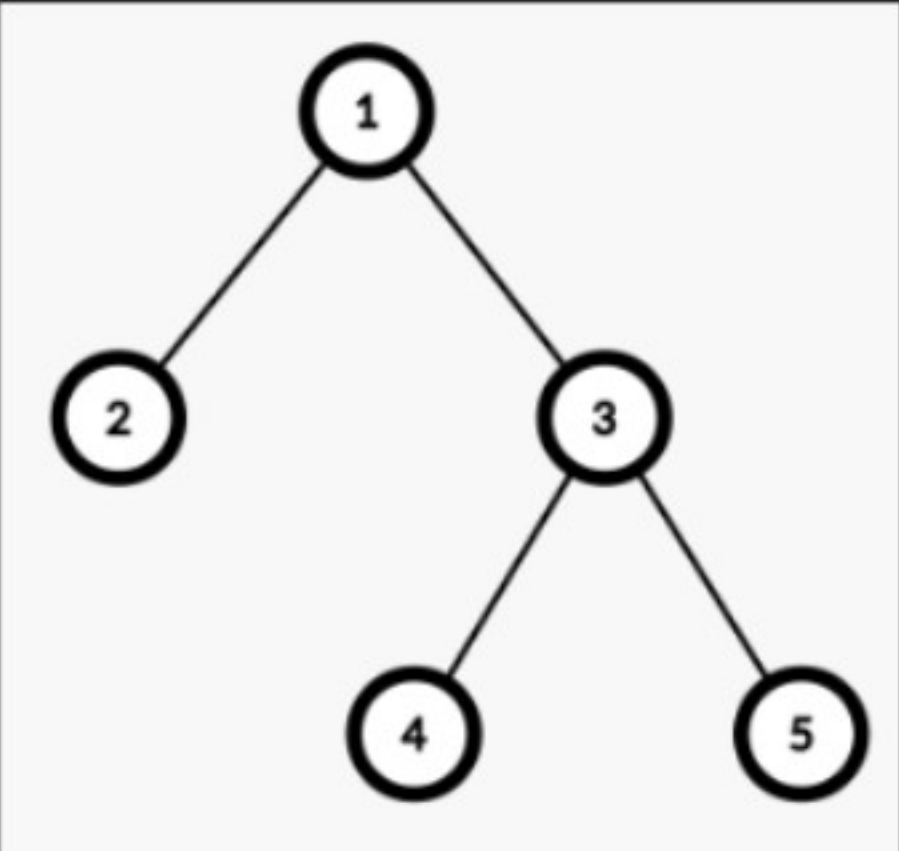
Input: `edges = [[1,2]]`, `queries = [[1,1],[1,2]]`

Output: `[0,1]`

Explanation:

- Query `[1,1]`: The path from Node 1 to itself consists of no edges, so the cost is 0. Thus, the number of valid assignments is 0.
- Query `[1,2]`: The path from Node 1 to Node 2 consists of one edge (`1 → 2`). Assigning weight 1 makes the cost odd, while 2 makes it even. Thus, the number of valid assignments is 1.

Example 2:



Input: `edges = [[1,2],[1,3],[3,4],[3,5]]`, `queries = [[1,4],[3,4],[2,5]]`

Output: `[2,1,4]`

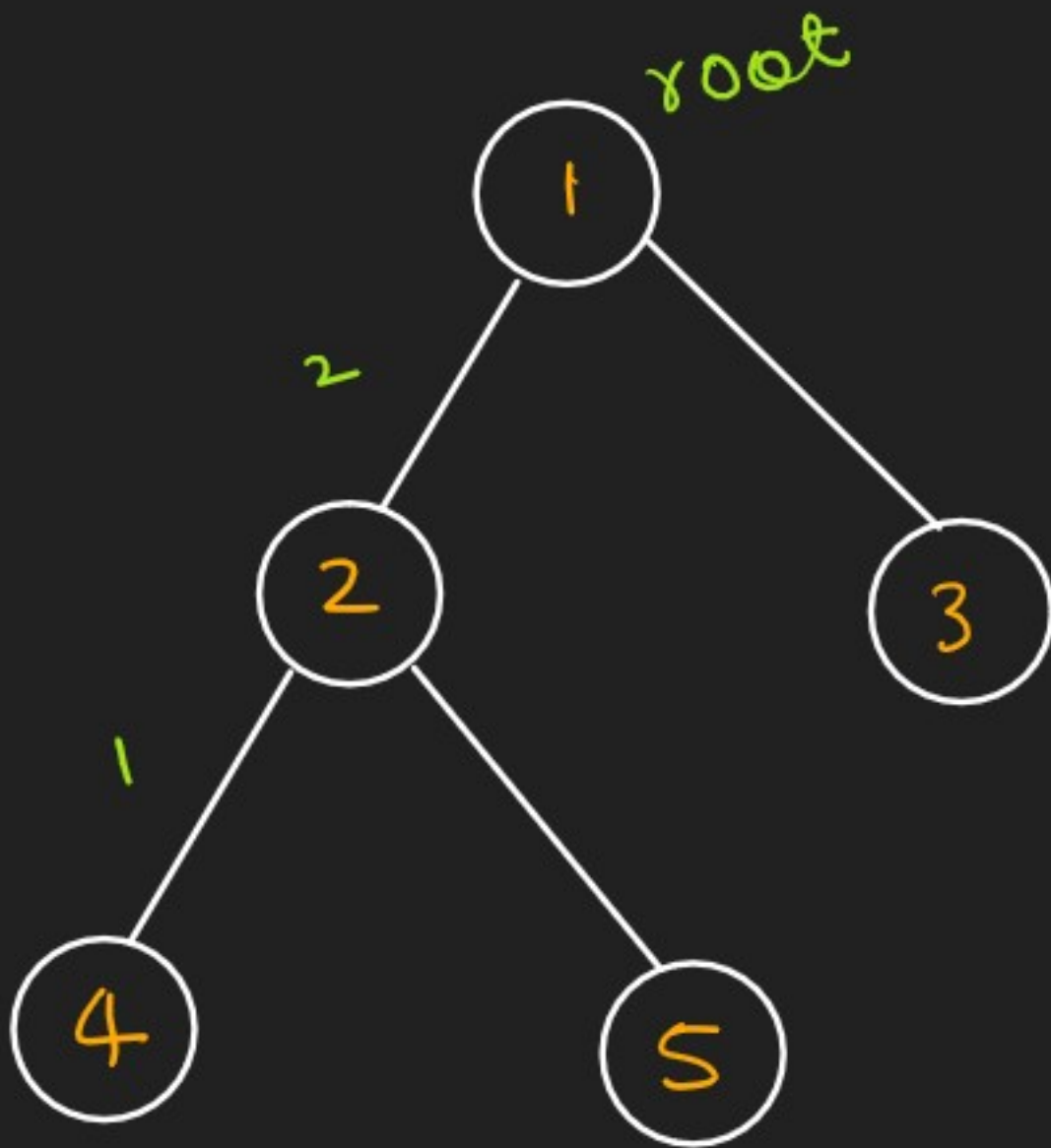
Explanation:

- Query `[1,4]`: The path from Node 1 to Node 4 consists of two edges (`1 → 3` and `3 → 4`). Assigning weights (1,2) or (2,1) results in an odd cost. Thus, the number of valid assignments is 2.
- Query `[3,4]`: The path from Node 3 to Node 4 consists of one edge (`3 → 4`). Assigning weight 1 makes the cost odd, while 2 makes it even. Thus, the number of valid assignments is 1.
- Query `[2,5]`: The path from Node 2 to Node 5 consists of three edges (`2 → 1`, `1 → 3`, and `3 → 5`). Assigning (1,2,2), (2,1,2), (2,2,1), or (1,1,1) makes the cost odd. Thus, the number of valid assignments is 4.

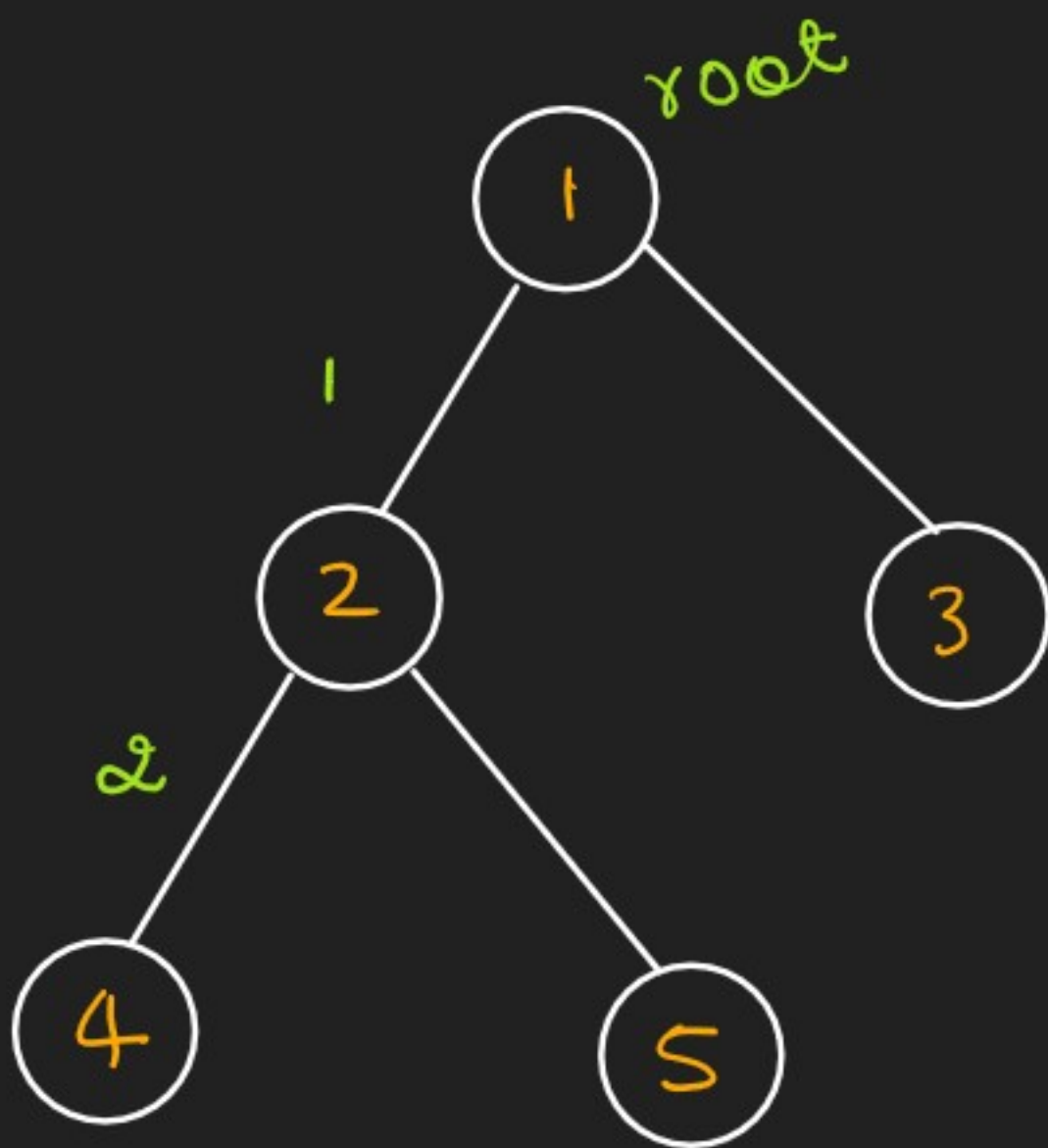
Constraints:

- $2 \leq n \leq 10^5$
- `edges.length == n - 1`
- `edges[i] == [ui, vi]`
- $1 \leq \text{queries.length} \leq 10^5$
- `queries[i] == [ui, vi]`
- $1 \leq u_i, v_i \leq n$
- `edges` represents a valid tree.

Thought Process



$$\begin{aligned}d &= 2 \\2^{d-1} &= 2^{2-1} \\&= 2^1 \\&= 2\end{aligned}$$

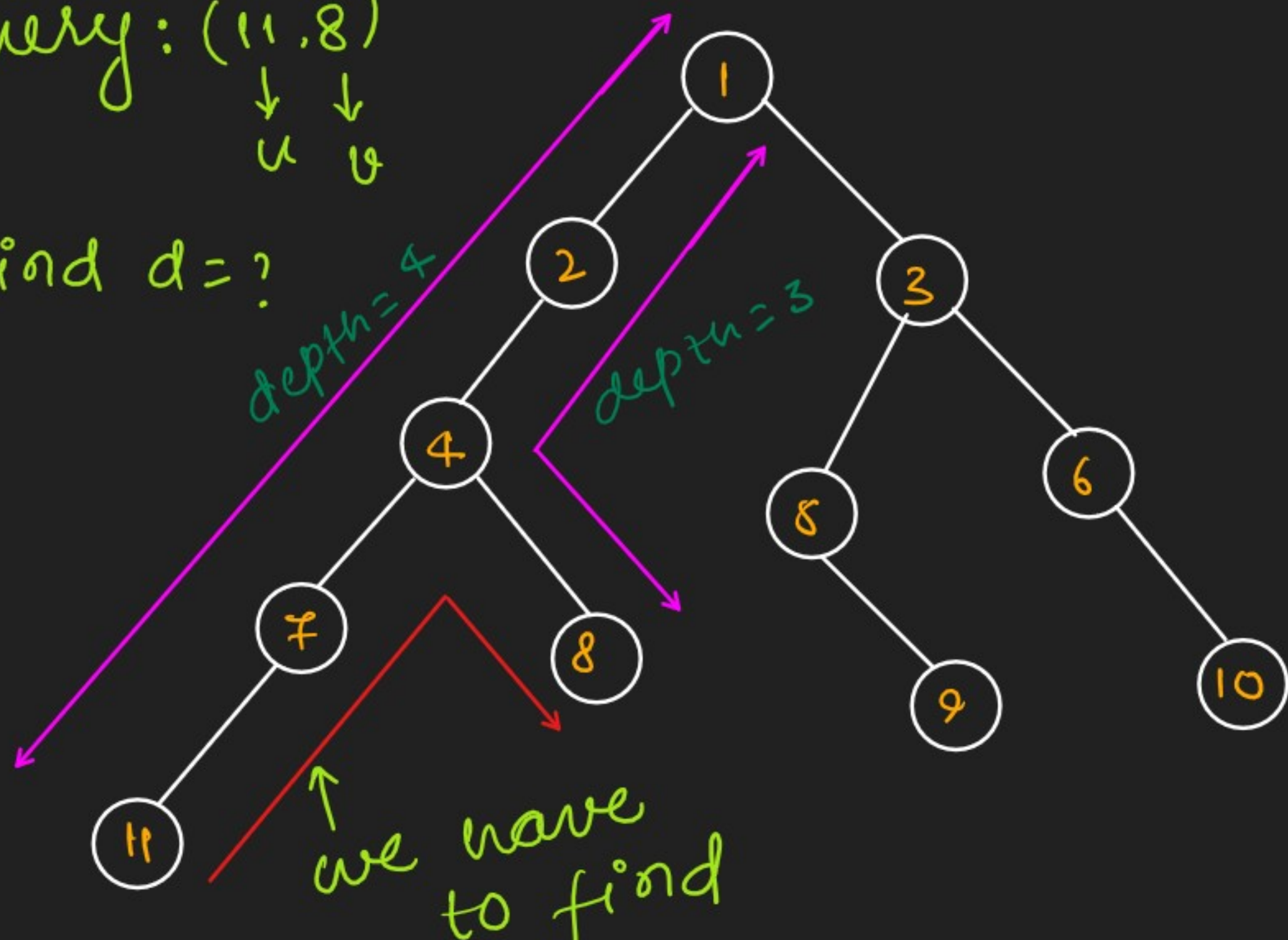


$$\begin{aligned}d &= 2 \\2^{d-1} &= 2^{2-1} \\&= 2^1 \\&= 2\end{aligned}$$

$(u, v): d \rightarrow 2^{d-1}$
↓
find optimally

Query: (11, 8)

find $d = ?$



$$\text{depth}[11] = 4$$

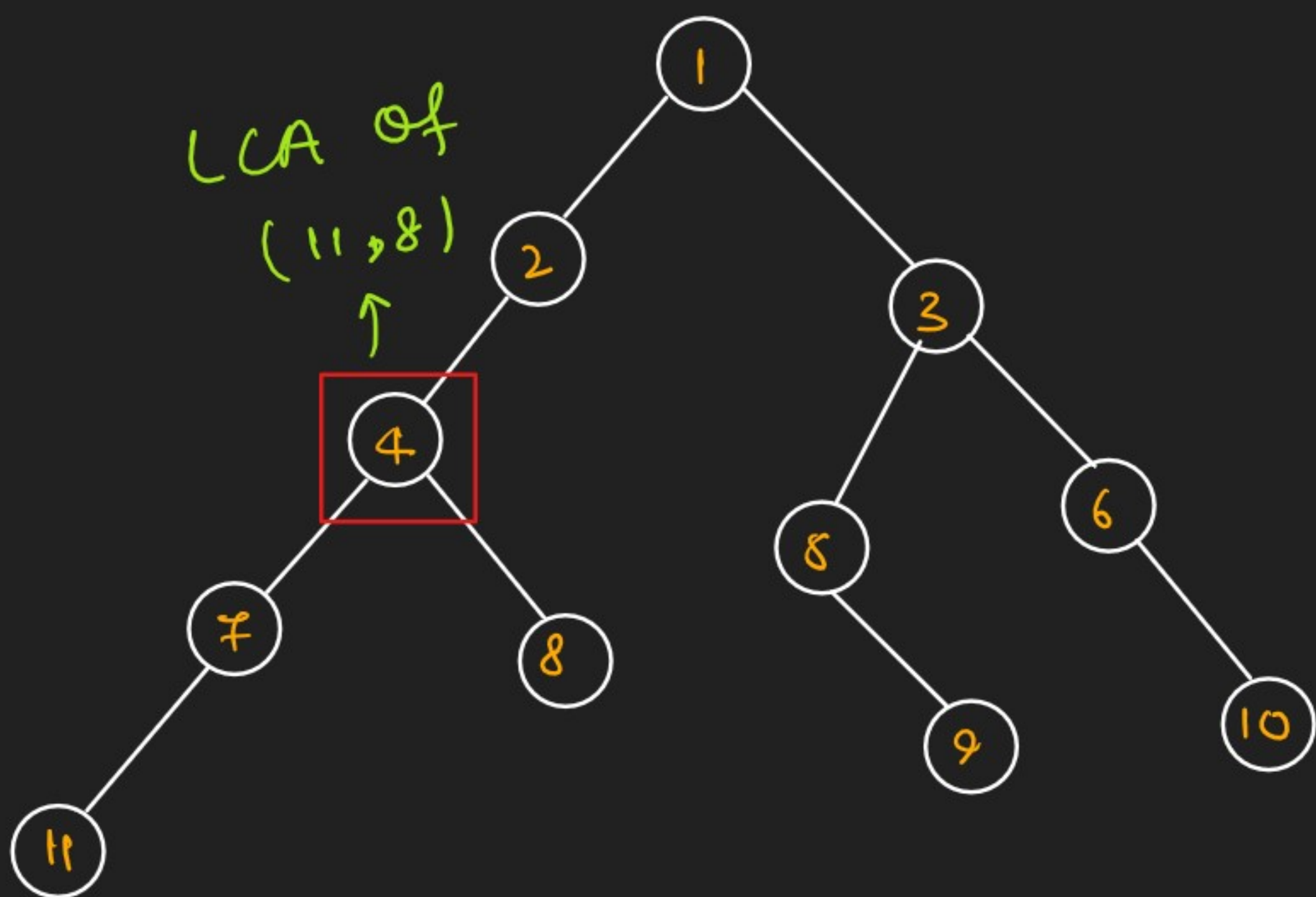
$$\text{depth}[3] = 3$$

$$\text{depth}[11] + \text{depth}[8] - \underbrace{2 * \text{depth}[4]}_{\text{because it has double.}}$$

$$* \text{depth}[u] + \text{depth}[v] + 2 * \text{depth}[?]$$

converging
↓
?

It is not but
lowest common
Ancestor



That means:

$$\text{depth}[u] + \text{depth}[v] = 2 * \text{depth}[\text{LCA}(u, v)]$$

converging node
(LCA)

Steps to code:

(-) one time: Find $\text{depth}[\text{node}] \rightarrow$ distance from root DFS.

(-) Query(u, v)

↳ $\text{depth}[u]$

↳ $\text{depth}[v]$

↳ $\text{LCA}(u, v)$

↳ $d = \text{depth}[u] + \text{depth}[v] - 2 \times \text{depth}[\text{LCA}];$

↳ $\text{result} = 2^{d-1};$

// corner case:

If (u, v) are in same path,

then who will LCA ??

Ans: $v \rightarrow$ will LCA $\checkmark$